

Практическое задание 1

Организация доступа к общим ресурсам

Мьютекс - объект синхронизации, который разрешает только одному потоку в один момент времени выполнять критический участок кода.

- защищает общие данные от гонок (race conditions) при доступе из нескольких потоков
- использовать когда несколько потоков читают/пишут в одну и ту же память

Семафор - счётчик, ограничивающий количество потоков, которые могут одновременно использовать ресурс.

- регулирует доступ к ограниченным ресурсам (буферы, соединения, лицензии)
- использовать когда нужно разрешить нескольким, но ограниченному числу потоков работать параллельно.

Условная переменная (condition variable) - механизм, позволяющий потоку ждать наступления события (например, «данные появились в буфере»).

- чтобы избежать активного ожидания (busy-waiting) и эффективно приостанавливать поток до нужного состояния.
- использовать в связке с мьютексом для задач типа Producer-Consumer, когда поток должен ждать, пока другое условие не станет истинным.

Producer-Consumer (производитель-потребитель) - классическая задача синхронизации: один поток производит данные, другой — потребляет.

- Нужна чтобы организовать безопасный обмен данными между потоками через общий буфер.
- использовать при построении очередей, конвейеров обработки, потоковых систем.

Инверсия приоритетов - ситуация, когда высокоприоритетный поток блокируется, потому что низкоприоритетный удерживает нужный ресурс, а средний мешает низкому освободить его.

Как решать:

- Наследование приоритета — низкий поток временно получает приоритет высокого.
- Приоритетная защита — поток выполняется с минимальным гарантированным приоритетом.

Сигналы - асинхронные уведомления от ОС процессу.

- чтобы обрабатывать внешние события - завершение, таймеры, пользовательские команды.
- обработчики должны быть очень простыми (только `async-signal-safe` функции: `write`, `_exit`).
- `SIGKILL` и `SIGSTOP` нельзя перехватить.

Таймер - механизм периодического или однократного уведомления по времени.

- чтобы выполнять действия по расписанию (опрос датчиков, тайм-ауты, периодическая синхронизация).
- по истечении времени ОС посылает сигнал (например, `SIGALRM`).

Менеджер ресурсов (`resource manager`) - компонент ОС, предоставляющий единый интерфейс к устройству или сервису через стандартные вызовы (`open`, `read`, `write`, `ioctl`).

- чтобы абстрагировать работу с устройствами, обеспечить безопасный и унифицированный доступ из разных процессов

Эти механизмы решают ключевые проблемы многопоточности:

- Гонки данных - мьютексы, семафоры.
- Неэффективное ожидание - условные переменные.
- Согласованность работы потоков - `Producer-Consumer`.
- Нарушение приоритетов - наследование приоритета.
- Реакция на внешние события - сигналы и таймеры.
- Унификация доступа к ресурсам - менеджер ресурсов.

Выполнение задания

1. hello.c

Программа бесконечно выводит переданные ей аргументы командной строки, нумеруя каждую строку и делая паузу в 1 секунду между выводами

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/intro/hello.c -o bin/hello
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/hello Привет мир
#0: Привет мир
#1: Привет мир
#2: Привет мир
#3: Привет мир
#4: Привет мир
#5: Привет мир
#6: Привет мир
#7: Привет мир
#8: Привет мир
#9: Привет мир
^C
nastya@Ubuntu:~/Systems real time/tasks/task1$
```

2. intro.c

Программа запускает 3 параллельных потока (чтение символов с клавиатуры, ожидание изменений, отображение текущего состояния). Потоки синхронизируются с помощью мьютекса и условной переменной, чтобы избежать конфликтов с доступом. Работает бесконечно.

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/intro
Hello World!

-----
r
-----

The state has changed! It is now in Ready State
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
n!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

The state has changed! It is now in Not Ready State
-----
^C
nastya@Ubuntu:~/Systems real time/tasks/task1$
```

3. nomutex.c

Код демонстрирует гонку данных при одновременном доступе нескольких потоков к общим переменным var1 и var2 без синхронизации. Программа выводит ошибки, потому что потоки нарушают условие `var1 == var2`

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/shared_mem/nomutex.c -o bin/nomutex
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/nomutex
nomutex: starting 16 threads...
nomutex: RACE DETECTED! Thread 0: var1=8492, var2=8492
nastya@Ubuntu:~/Systems real time/tasks/task1$
```

Я изменила изначальный код, так как у меня не возникала проблема гонки и теперь:

- без `do_work()` проверка выполняется на каждой итерации цикла - резко повышает вероятность обнаружения несогласованности
- после создания потоков в `main()` добавлен вызов `sleep(2)`, чтобы потоки успели выполнить операции
- убрана строка `pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);` - предотвращает мгновенное уничтожение потоков до начала их работы

4. mutex.c

После добавления мьютекса вокруг операций `var1++` и `var2++` доступ становится последовательным, гонка устраняется, и переменные всегда остаются равными

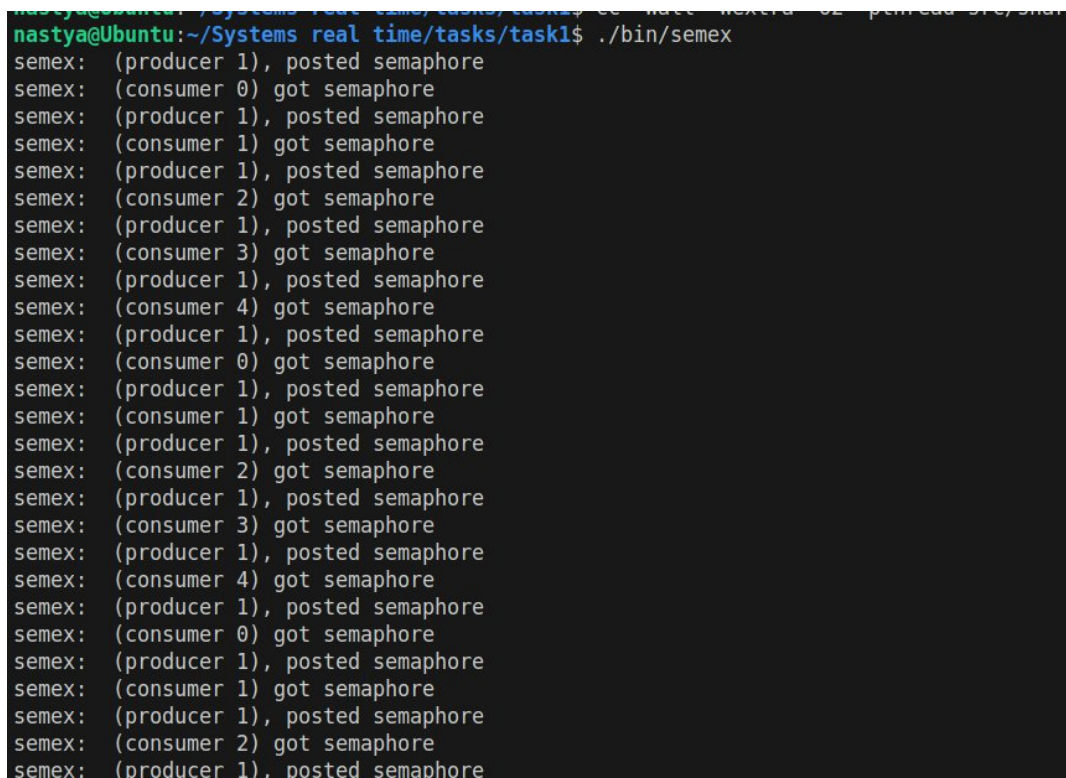
```
void *update_thread(void *arg) {
    long id = (long)arg;
    while (!stop_flag) {
        pthread_mutex_lock(&mutex);
        var1++;
        var2++;
        if (var1 != var2) {
            printf("Ошибка\n");
            exit(1);
        }
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/mutex
mutex: starting 16 threads...
mutex: all done. var1 = 42830838, var2 = 42830838
nastya@Ubuntu:~/Systems real time/tasks/task1$
```

5. semex.c

Код демонстрирует синхронизацию потоков с помощью POSIX семафора

1. Создается семафор со значением 0 — все потоки, которые его ждут, блокируются
2. Запускаются 5 потоков потребителей, каждый вызывает `sem_wait()` и ждёт
3. Один поток производитель каждую секунду вызывает `sem_post()`, увеличивая счётчик семафора на 1
4. Это разблокирует один из ждущих потребителей, который выводит сообщение и снова ждёт
5. Каждую секунду один случайный потребитель получает разрешение на выполнение



```
nasty@Ubuntu:~/Systems real time/tasks/task1$ ./bin/semex
semex: (producer 1), posted semaphore
semex: (consumer 0) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 1) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 2) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 3) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 4) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 0) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 1) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 2) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 3) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 4) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 0) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 1) got semaphore
semex: (producer 1), posted semaphore
semex: (consumer 2) got semaphore
semex: (producer 1), posted semaphore
```

6. prodcons.c

Код решает задачу «производитель–потребитель» с помощью условной переменной и мьютекса.

Есть 1 ячейка памяти (product) и флаг состояния (state):

state = 0 - пусто, можно производить

state = 1 - полный, можно потреблять

Производитель- ждет пока ячейка не освободится, кладет новое значение, сигнализирует потребителю.

Потребитель - ждет пока появятся данные, забирает их, сигнализирует производителю.

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/prodcons
prodcons: produced 2, state 0
prodcons: consumed 2, state 1
prodcons: produced 4, state 0
prodcons: consumed 4, state 1
prodcons: produced 6, state 0
prodcons: consumed 6, state 1
prodcons: produced 8, state 0
prodcons: consumed 8, state 1
prodcons: produced 10, state 0
prodcons: consumed 10, state 1
prodcons: produced 12, state 0
prodcons: consumed 12, state 1
prodcons: produced 14, state 0
prodcons: consumed 14, state 1
prodcons: produced 16, state 0
prodcons: consumed 16, state 1
prodcons: produced 18, state 0
prodcons: consumed 18, state 1
prodcons: produced 20, state 0
prodcons: consumed 20, state 1
prodcons: produced 22, state 0
prodcons: consumed 22, state 1
prodcons: produced 24, state 0
prodcons: consumed 24, state 1
prodcons: produced 26, state 0
prodcons: consumed 26, state 1
```

7. condvar.c

Код реализует машину состояний из 4 состояний с помощью 4 потоков и синхронизации через мьютекс и условную переменную

Состояние 0 - всегда переходит в 1 и увеличивает счётчик

Состояние 1 - переходит в 2, если счётчик чётный, или в 3, если нечётный

Состояния 2 и 3 - оба возвращаются в 0

Потоки ждут своей очереди, и только тот, чьё состояние наступило, выполняет переход

Добавлены обработчики состояний 2 и 3.

```
71
72 //обработчик состояния 2 - переходит в 0
73 void *state_2(void *arg) {
74     while (1) {
75         pthread_mutex_lock(&mutex);
76         while (state != 2) {
77             pthread_cond_wait(&cond, &mutex);
78         }
79         printf("%s: transit 2 -> 0\n", proname);
80         state = 0;
81         pthread_cond_broadcast(&cond);
82         pthread_mutex_unlock(&mutex);
83     }
84     return NULL;
85 }
86
87 //обработчик состояния 3 - переходит в 0
88 void *state_3(void *arg) {
89     while (1) {
90         pthread_mutex_lock(&mutex);
91         while (state != 3) {
92             pthread_cond_wait(&cond, &mutex);
93         }
94         printf("%s: transit 3 -> 0\n", proname);
95         state = 0;
96         pthread_cond_broadcast(&cond);
97         pthread_mutex_unlock(&mutex);
98     }
99     return NULL;
100 }
```

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/condvar
condvar: transit 0 -> 1
condvar: transit 1 -> 3
condvar: transit 3 -> 0
condvar: transit 0 -> 1
condvar: transit 1 -> 2
condvar: transit 2 -> 0
condvar: transit 0 -> 1
condvar: main, exiting
nastya@Ubuntu:~/Systems real time/tasks/task1$
```

8. working.c

Код демонстрирует проблему инверсии приоритетов в многопоточных системах. Её можно решить с помощью наследования приоритета.

Без наследования приоритета высокоприоритетный поток вынужден ждать, пока низкоприоритетный освободит ресурс, но тот не может работать, потому что средний поток забирает процессорное время — возникает инверсия приоритета

9. scenario_1.c

Наблюдаем проблему инверсии приоритетов

T2 (приоритет 30) должен был бы выполняться сразу, но ждал T1 (приоритет 20). T1 не нуждался в мьютексе, но косвенно блокировал T2, мешая SERVER освободить ресурс

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/inv_prio/working.c src/inv_prio/scenario_1.c -o bin/inv_s1
nastya@Ubuntu:~/Systems real time/tasks/task1$ sudo ./bin/inv_s1
[SERVER] стартует и захватывает ресурс
server is working for 0 - 0th start
[T2 high] пытается получить ресурс
[T1 mid] стартует (фоновая нагрузка)
server is working for 0 - 0th end
server is working for 0 - 1th start
server is working for 0 - 1th end
server is working for 0 - 2th start
server is working for 0 - 2th end
server is working for 0 - 3th start
server is working for 0 - 3th end
server is working for 0 - 4th start
server is working for 0 - 4th end
server finished all work for 0
[SERVER] освободил ресурс
[T2 high] получил ресурс
[T2 high] освободил ресурс и завершился
[T1 mid] завершился
scenario 1: завершено. Наблюдайте временную задержку у t2 из-за t1 (инверсия приоритетов).
```

10. scenario_2.c

Код демонстрирует решение проблемы инверсии приоритета с помощью мьютекса с наследованием приоритета (PTHREAD_PRIO_INHERIT)

```
nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/inv_prio/working.c src/inv_prio/scenario_2.c -o bin/inv_s2
nastya@Ubuntu:~/Systems real time/tasks/task1$ sudo ./bin/inv_s2
[SERVER] стартует и захватывает ресурс
server is working for 0 - 0th start
server is working for 0 - 0th end
server is working for 0 - 1th start
server is working for 0 - 1th end
server is working for 0 - 2th start
server is working for 0 - 2th end
server is working for 0 - 3th start
[T2 high] пытается получить ресурс
[T1 mid] стартует (фоновая нагрузка)
server is working for 0 - 3th end
server is working for 0 - 4th start
server is working for 0 - 4th end
server finished all work for 0
[SERVER] освободил ресурс
[T2 high] получил ресурс
[T2 high] освободил ресурс и завершился
[T1 mid] завершился
scenario 2 завершено
```

Сравнение с scenario_1:

- Без наследования: T2 ждал ВСЮ работу SERVER (5 итераций с прерываниями от T1)

- С наследованием: T2 ждал только 2 итерации SERVER без прерываний

11. intsimple.c

Код демонстрирует обработку сигналов и ввода с клавиатуры

1. Настраивает обработку сигналов

SIGINT - Ctrl+C в терминале

SIGTERM - «вежливый» запрос на завершение

SIGUSR1, SIGUSR2 - пользовательские сигналы

SIGKILL - нельзя перехватить

Каждый сигнал устанавливает флаг (volatile sig_atomic_t), чтобы основной цикл знал, что произошло событие

2. Включает неблокирующий ввод с клавиатуры (raw режим терминала)

3. Работает в бесконечном цикле, пока не нажата 'q'

```

nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/interrupt/intsimple.c -o bin/intsimple
nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/intsimple
intsimple: starting...
Поддерживаемые сигналы: SIGINT(Ctrl+C), SIGTERM, SIGUSR1, SIGUSR2.
Замечание: SIGKILL нельзя перехватить или обработать на Linux.
Нажмите 'q' для выхода.
intsimple: получен SIGINT (Ctrl+C)
intsimple: получен SIGTERM
intsimple: клавиша 'a'
intsimple: получен SIGUSR1

```

```

nastya@Ubuntu:~/Systems real time/tasks/task1$ pidof intsimple
26704
nastya@Ubuntu:~/Systems real time/tasks/task1$ kill -TERM 26704
nastya@Ubuntu:~/Systems real time/tasks/task1$ kill -USR1 26704
nastya@Ubuntu:~/Systems real time/tasks/task1$

```

12. int.c

Код демонстрирует, как в Unix системах можно эмулировать поведение аппаратных прерываний с помощью:

- сигналов (SIGALRM),

- интервального таймера (setitimer),
- безопасного обработчика (async-signal-safe функции)

Программа печатает "100 events" каждую секунду и завершается через 10 секунд

```
● nasty@Ubuntu:~/Systems real time/tasks/task1$ ./bin/int
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
100 events
○ nasty@Ubuntu:~/Systems real time/tasks/task1$
```

Мне пришлось изменить изначальный код, так как он ничего не выводил:

- функция `sleep()` прерывалась при получении сигнала и не возобновлялась автоматически – используем надёжное ожидание через цикл с `time()`
- При таймере 10 мс сигналы приходили слишком часто, и `sleep()` постоянно прерывался и программа заканчивалась почти сразу – используем цикл с `sleep(1)` и проверкой времени

```

18  int main(void) {
19      struct sigaction sa = {0};
20      sa.sa_handler = on_alarm;
21      sigemptyset(&sa.sa_mask);
22      sigaction(SIGALRM, &sa, NULL);
23
24      struct itimerval t = {{0, 10000}, {0, 10000}};
25      setitimer(ITIMER_REAL, &t, NULL);
26
27      //ожидание 20 секунд
28      time_t start = time(NULL);
29      while (time(NULL) - start < 20) {
30          sleep(1);
31      }
32
33      t.it_value.tv_sec = t.it_value.tv_usec = 0;
34      setitimer(ITIMER_REAL, &t, NULL);
35
36      return 0;
37  }

```

13. resmgr.c

Запускает сервер на сокете /tmp/example_resmgr.sock

Принимает подключения и создаёт отдельный поток для каждого клиента

Обработывает сигналы SIGINT и SIGTERM для корректного завершения (удаление файла сокета)

Устройство хранит одно целое число (device_value) как своё состояние

Поддерживает три текстовые команды:

GET — вернуть текущее значение.

SET <N> — установить новое значение N.

INC — увеличить значение на 1.

Только первый подключившийся клиент получает права на запись (SET, INC). Все последующие клиенты могут только читать (GET)

Общий ресурс (device_value) защищён мьютексом (pthread_mutex_t), что гарантирует корректную работу в многопоточной среде

Обработчики сигналов используют _exit() для безопасного завершения без вызова деструкторов

14. client.c

Простая утилита командной строки для взаимодействия с менеджером ресурсов.

Использование:

`./client "GET"`

`./client "SET 42"`

`./client "INC"`

Автоматически добавляет символ новой строки (`\n`) к команде — необходимо для корректного распознавания команды сервером

Отправляет команду и выводит полученный ответ. Поддерживает те же команды, что и сервер: GET, SET N, INC

Проверяет ошибки при создании сокета, подключении, отправке и приёме данных. Корректно завершает соединение после получения ответа.

```
bash: ./src/client: не такой файл или каталог
● nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/resmgr_client "SET 100"
OK
● nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/resmgr_client "GET"
100

● nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/resource_manager/resmgr.c -o bin/resmgr
● nastya@Ubuntu:~/Systems real time/tasks/task1$ cc -Wall -Wextra -O2 -pthread src/resource_manager/client.c -o bin/resmgr_client
○ nastya@Ubuntu:~/Systems real time/tasks/task1$ ./bin/resmgr
example: starting...
example: listening on /tmp/example_resmgr.sock
Примеры команд:
echo 'GET' | nc -U /tmp/example_resmgr.sock
echo 'SET 42' | nc -U /tmp/example_resmgr.sock
echo 'INC' | nc -U /tmp/example_resmgr.sock
□
```

1. Первый клиент успешно установил значение 100. Он получил права на запись.
2. Любой клиент (включая последующих) может читать - значение 100 возвращается корректно
3. Второй (и все последующие) клиенты не имеют прав на запись.